

Part 2 请求分页存储管理



学习目标

- 能够理解请求分页的实现原理
- 理解缺页中断的实现过程
- 能够理解并应用请求分页的地址转换过程
- 能够分析并应用页面置换算法
- 能够分析影响请求分页性能的因素及解决方法

Part 2 请求分页存储管理

2.1 基本原理

请求分页要解决的问题

- ❖ 作业是否可以装入部分运行？——局部性原理
- ❖ 运行之后，若访问到没有装入的作业，如何发现？
 - 修改进程页表
 - 增加缺页中断
- ❖ 需要将不在内存的作业装入内存，内存已满如何装入？如何调出？
 - 置换算法

Part 2 请求分页存储管理

2.2 硬件支持

页表机制：需要在进程页表中添加若干项

- 状态位P：指示该页是否在内存
- 访问字段A：记录该页在一段时间内被访问的次数
- 修改位M：也称脏位，标志该页是否被修改过
- 外存地址：指示该页在外存中的地址（物理块号）

页号	物理块号	状态位P	访问字段A	修改位M	外存地址
----	------	------	-------	------	------

2.2 硬件支持

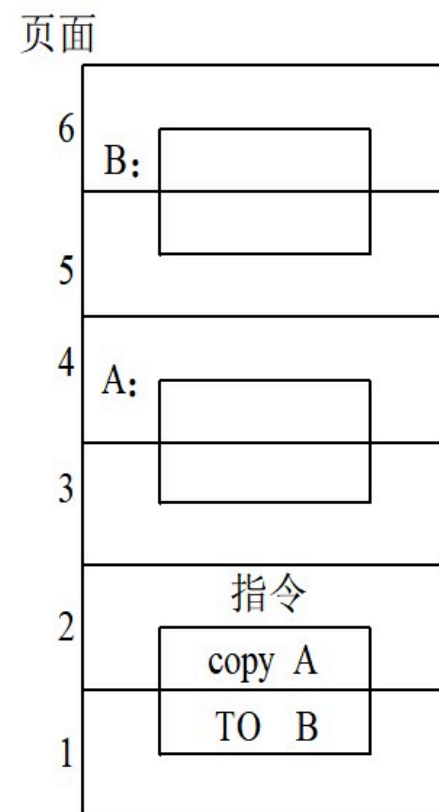
缺页中断机构

- 在指令执行期间产生和处理中断信号
- 一条指令在执行期间，可能产生多次缺页中断

Part 2 请求分页存储管理

2.2 硬件支持

- 缺页中断的特殊性
- 缺页中断在指令执行期间产生和进行处理而不是在一条指令执行完毕之后。所缺的页面调入之后，重新执行被中断的指令。
- 一条指令的执行可能产生多次缺页中断，如：COPY A, B若指令本身和两个操作数A, B都跨越相邻外存页的分界处，则产生6次缺页中断。



2.2 硬件支持

地址变换机构

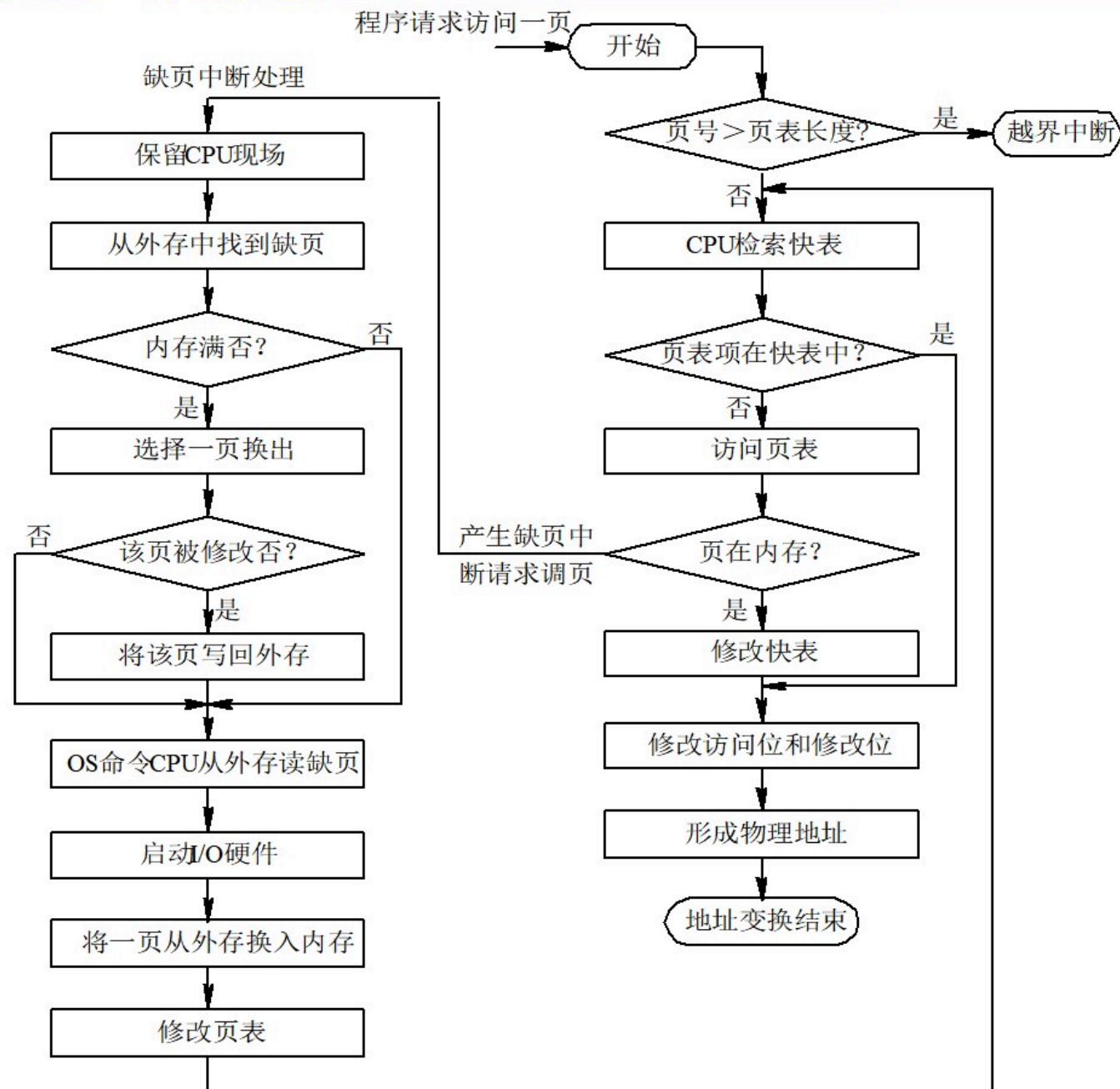
➤ 与分页内存管理方式类似

- 由逻辑地址检索页表，根据对应页表项的标志位判断，若在内存中，则直接形成物理地址。
- 如不在内存中，产生缺页中断，从对应页表项的外存地址，从外存找到该页，内存满，则选择页面换出，否则从外存直接调入内存。完成地址转换。

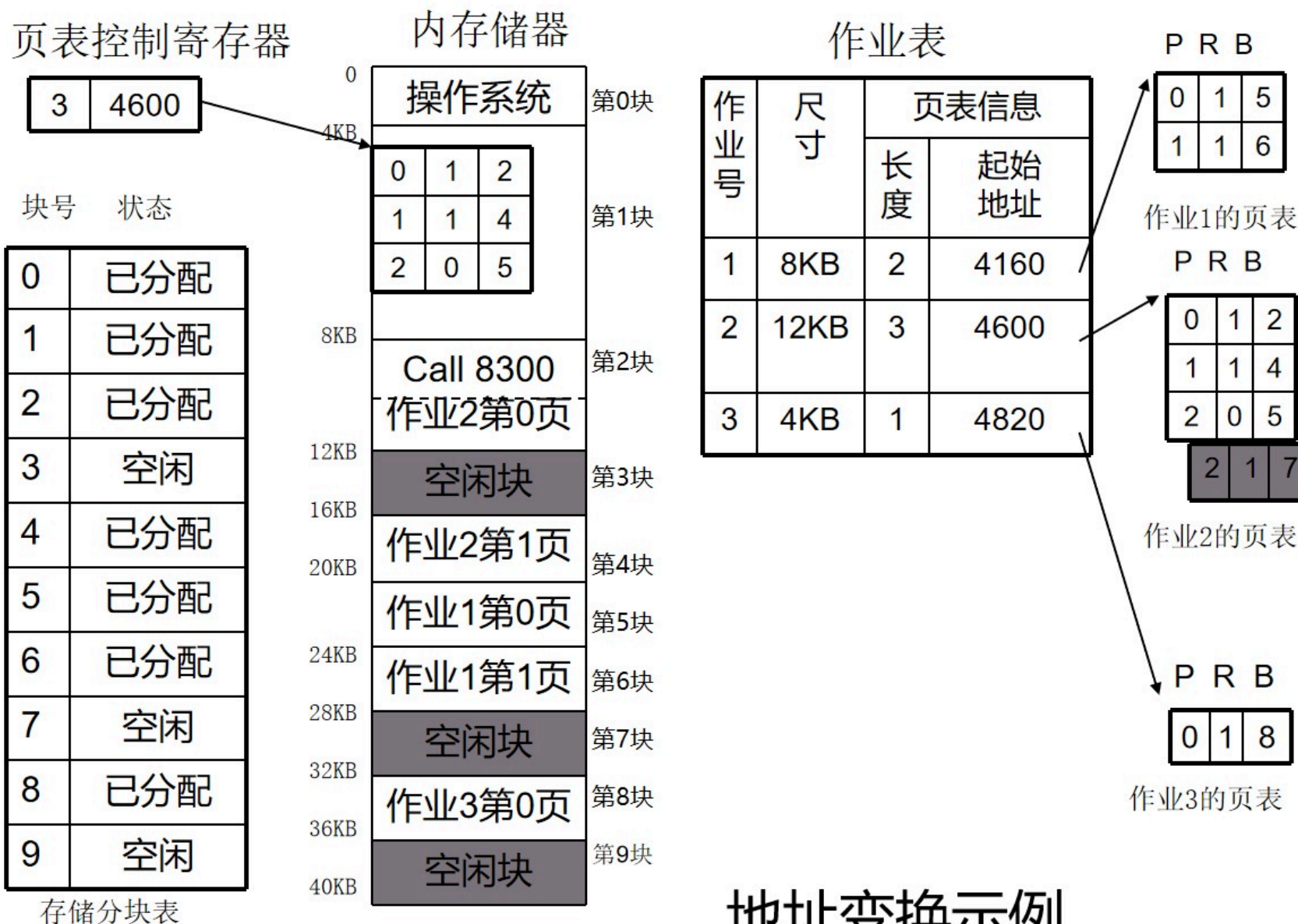
Part 2 请求分页存储管理

2.2

地址变换



Part 2 请求分页存储管理



Part 2 请求分页存储管理

2.3 内存分配



最小物理块数的确定：

- 保证进程正常运行所需的最小物理块数。



物理块的分配策略：

- 固定分配 局部置换；
- 可变分配 全局置换；
- 可变分配 局部置换。



物理块分配算法：

- 平均分配算法；
- 按比例分配算法；
- 考虑优先权的分配算法；

□ 计算各进程页面总和S

$$S = \sum_{i=1}^n S_i$$

□ 计算每个进程所能分到的物理块数 b_i

$$b_i = \frac{S_i}{S} \times m$$

2.4 调页策略



何时调入页面

- 预调页策略：预先调入一些页面到内存
- 请求调页策略：发现需要访问的页面不在内存时，调入内存



从何处调入页面

- 如系统拥有足够的对换区空间，全部从对换区调入所需页面
- 如系统缺少足够的对换区空间，凡是不会被修改的文件，都直接从文件区调入；当换出这些页面时，由于未被修改而不必再将它们重写磁盘，以后再调入时，仍从文件区直接调入
- UNIX方式：未运行过的页面，从文件区调入；曾经运行过但又被换出的页面，从对换区调入



如何调入页面？

2.4 调页策略

页面调入方法

- 01 查找所需页在磁盘上的位置。
- 02 查找一内存空闲块：
 - 如果有空闲块，就直接使用它；
 - 如果没有空闲块，使用页面置换算法选择一个“牺牲”内存块；
 - 将“牺牲”块的内容写到磁盘上，更新页表和物理块表。
- 03 将所需页读入（新）空闲块，更新页表。
- 04 重启用户进程。

Part 2 请求分页存储管理

2.4 调页策略

缺页率

访问页面成功(在内存)的次数为S

访问页面失败(不在内存)的次数为F

总访问次数为 $A=S+F$

缺页率为 $f= F/A$

影响因素：页面大小、分配内存块的数目、页面置换算法、程序固有属性

缺页中断处理的时间

$$t = \beta \times t_a + (1 - \beta) \times t_b$$

Part 2 请求分页存储管理

2.4 调页策略

缺页中断处理时间的例子

 存取内存的时间= 200 nanoseconds (ns)

 平均缺页处理时间 = 8 milliseconds (ms)


$$\begin{aligned} t &= (1 - p) \times 200\text{ns} + p \times 8\text{ms} \\ &= (1 - p) \times 200\text{ns} + p \times 8,000,000\text{ns} \\ &= 200\text{ns} + p \times 7,999,800\text{ns} \end{aligned}$$

 如果每1,000次访问中有一个缺页中断, 那么:

$$t = 8200 \text{ ns}$$

这是导致计算机速度放慢40倍的影响因子!

2.5 页面置换算法

功能：需要调入页面时，选择内存中哪个物理页面被置换。称为replacement policy。

目标：把未来不再使用的或短期内较少使用的页面调出，通常只能在局部性原理指导下依据过去的统计数据预测；

Part 2 请求分页存储管理

2.5 页面置换算法

最佳算法(OPT, optimal)

先进先出算法(FIFO)

最近最久未使用算法(LRU, Least Recently Used)

轮转算法(clock)

最不常用算法(LFU, Least Frequently Used)

页面缓冲算法(page buffering)

Part 2 请求分页存储管理

2.5 页面置换算法

最佳置换算法OPT



被置换的页将是之后最长时间不被使用的页

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		2			2			2				7		
	0	0	0		0		4			0			0				0		
		1	1		3		3			3			1				1		

page frames



怎样知道被置换的页是之后最长时间不被使用的页？



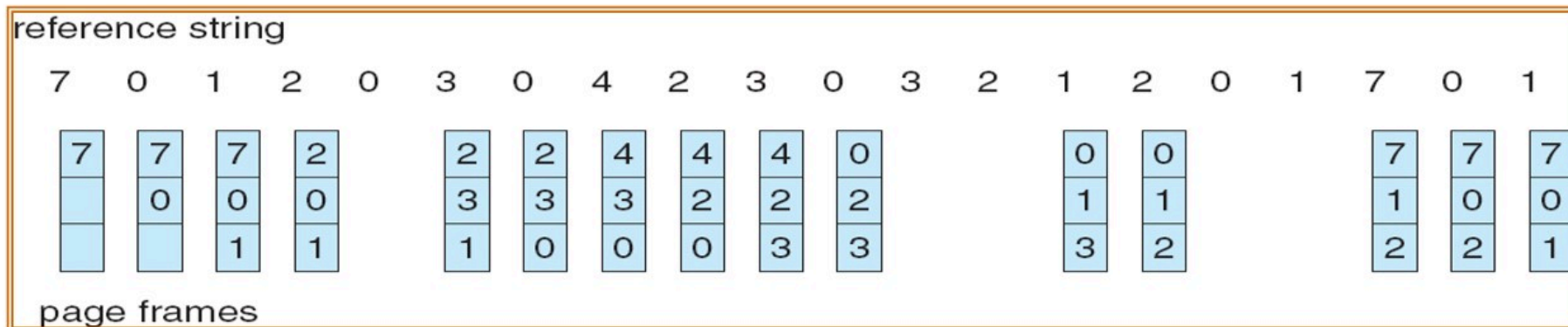
无法实现的算法，可用来评价其他算法

Part 2 请求分页存储管理

2.5 页面置换算法

先进先出置换算法

总是淘汰最先进入内存的页面，即选择在内存中驻留时间最久的页面予以淘汰



2.5 页面置换算法

先进先出置换算法

性能较差。较早调入的页往往是经常被访问的页，这些页在FIFO算法下被反复调入和调出。并且有Belady现象。

Q: 进程分配的物理块数越多，缺页情况如何变化？

2.5 页面置换算法

先进先出置换算法

- ❖ **Belady现象**：采用FIFO算法时，如果对一个进程未分配它所要求的全部页面，有时就会出现分配的页面数增多，缺页率反而提高的异常现象。
- ❖ **Belady现象的描述**：一个进程P要访问M个页，OS分配N个内存页面给进程P；对一个访问序列S，发生缺页次数为 $PE(S, N)$ 。当N增大时， $PE(S, N)$ 时而增大，时而减小。
- ❖ **Belady现象的原因**：FIFO算法的置换特征与进程访问内存的动态特征是矛盾的，即被置换的页面并不是进程不会访问的。

Part 2 请求分页存储管理

2.5 页面置换算法

先进先出置换算法

Belady现象的例子

❖ 进程P有5页程序访问页的顺序为：1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5;

❖ 如果在内存中分配3个页面，则缺页情况如下：12次访问中有缺页9次；

FIFO	1	2	3	4	1	2	5	1	2	3	4	5
页 0	1	1	1	4	4	4	5	5	5	5	5	5
页 1		2	2	2	1	1	1	1	1	3	3	3
页 2			3	3	3	2	2	2	2	2	4	4
缺 页	x	x	x	x	x	x	x	√	√	x	x	√

Part 2 请求分页存储管理

2.5 页面置换算法

先进先出置换算法

Belady现象的例子

❖ 如果在内存中分配4个页面，则缺页情况如下：12次访问中有缺页10次

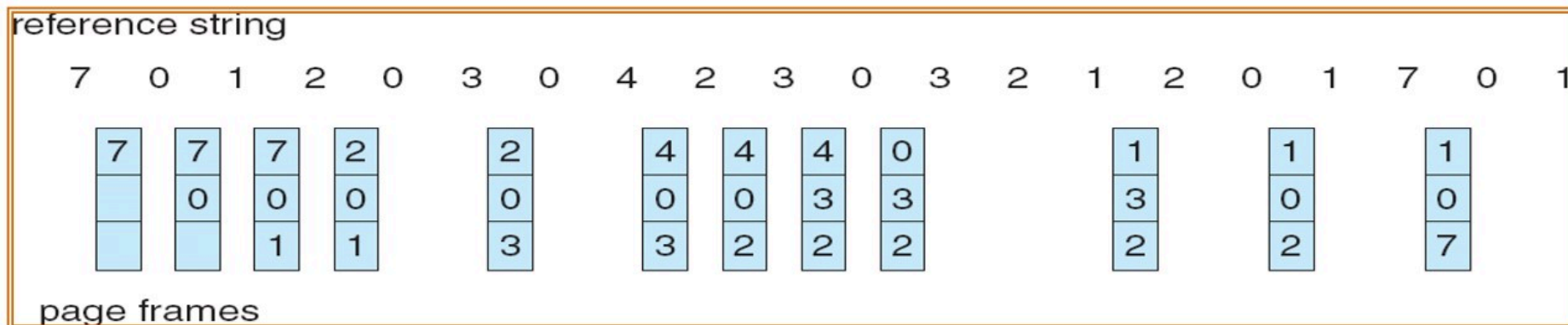
FIFO	1	2	3	4	1	2	5	1	2	3	4	5
页 0	1	1	1	1	1	1	5	5	5	5	4	4
页 1		2	2	2	2	2	2	1	1	1	1	5
页 2			3	3	3	3	3	3	2	2	2	2
页 3				4	4	4	4	4	4	3	3	3
缺页	x	x	x	x	√	√	x	x	x	x	x	x

Part 2 请求分页存储管理

2.5 页面置换算法

最近最久未使用算法(LRU)

选择内存中**最久未使用的页面**被置换。这是局部性原理的合理近似，性能接近最佳算法。但由于需要记录页面使用时间的先后关系，硬件开销太大。



2.5 页面置换算法

最近最久未使用算法(LRU)

硬件支持

❖ 寄存器

为了记录某进程在内存中各页的使用情况，须为每个在内存中的页面配置一个移位寄存器，可表示为

$$R=R_{n-1}R_{n-2}R_{n-3}\cdots R_2R_1R_0$$

Part 2 请求分页存储管理

2.5 页面置换算法

最近最久未使用算法(LRU)

硬件支持

某进程具有8个页面时的LRU访问情况

❖ 寄存器

实页 \ R	R ₇	R ₆	R ₅	R ₄	R ₃	R ₂	R ₁	R ₀
1	0	1	0	1	0	0	1	0
2	1	0	1	0	1	1	0	0
3	0	0	0	0	0	1	0	0
4	0	1	1	0	1	0	1	1
5	1	1	0	1	0	1	1	0
6	0	0	1	0	1	0	1	1
7	0	0	0	0	0	1	1	1
8	0	1	1	0	1	1	0	1

Part 2 请求分页存储管理

2.5 页面置换算法

最近最久未使用算法(LRU)

硬件支持

❖ 栈

4	7	0	7	1	0	1	2	1	2	6
							2	1	2	6
				1	0	1	1	2	1	2
		0	7	7	1	0	0	0	0	1
	7	7	0	0	7	7	7	7	7	0
4	4	4	4	4	4	4	4	4	4	7

用栈保存当前使用页面时栈的变化情况

2.5 页面置换算法

Clock置换算法

LRU的近似算法，又称最近未用(NRU)或二次机会页面置换算法

简单的Clock算法

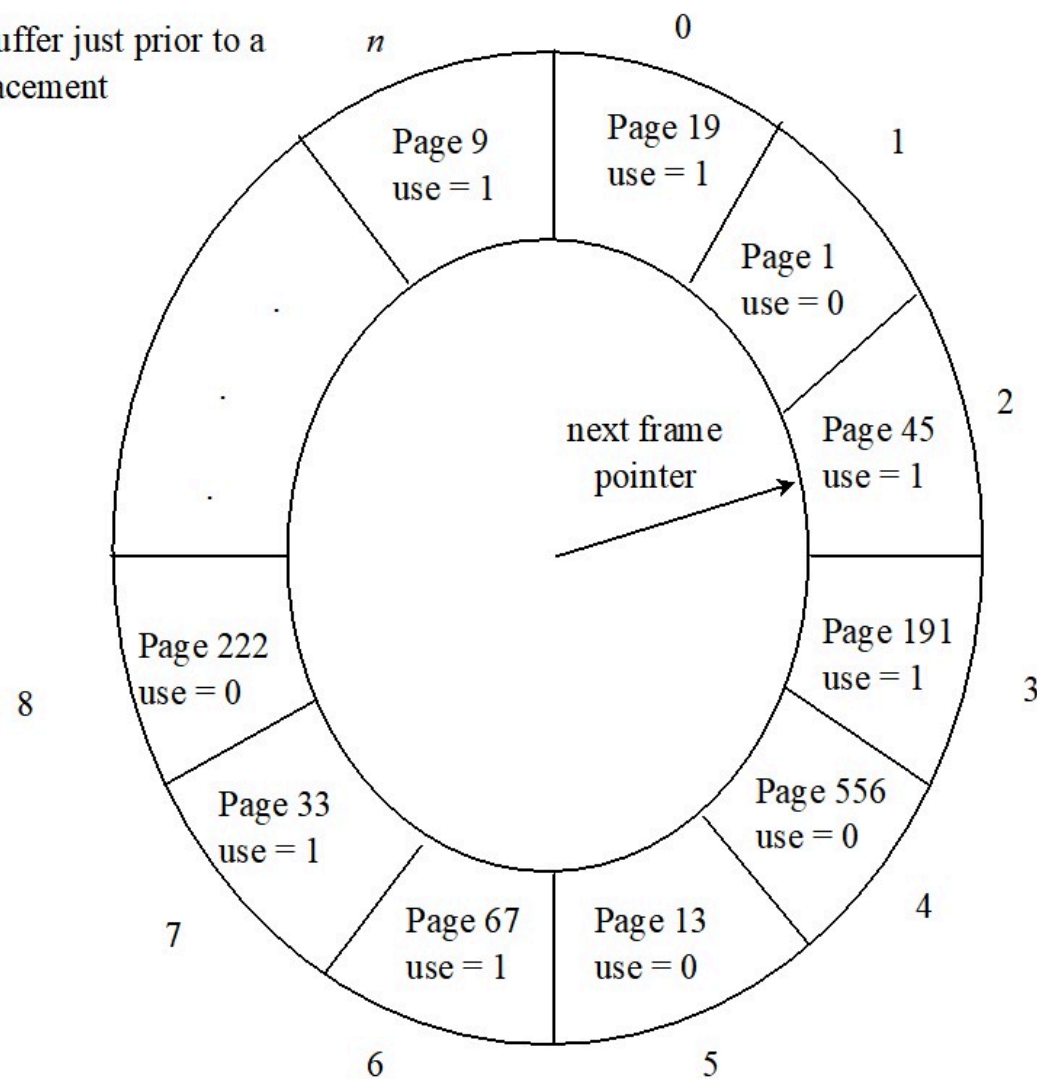
- 每个页都与一个访问位相关联，初始值为0
- 当页被访问时置访问位为1
- 置换时选择访问位为0的页；若为1，重新置为0

Part 2 请求分页存储管理

2.5 页面置换算法

State of buffer just prior to a
page replacement

Clock置换算法示例

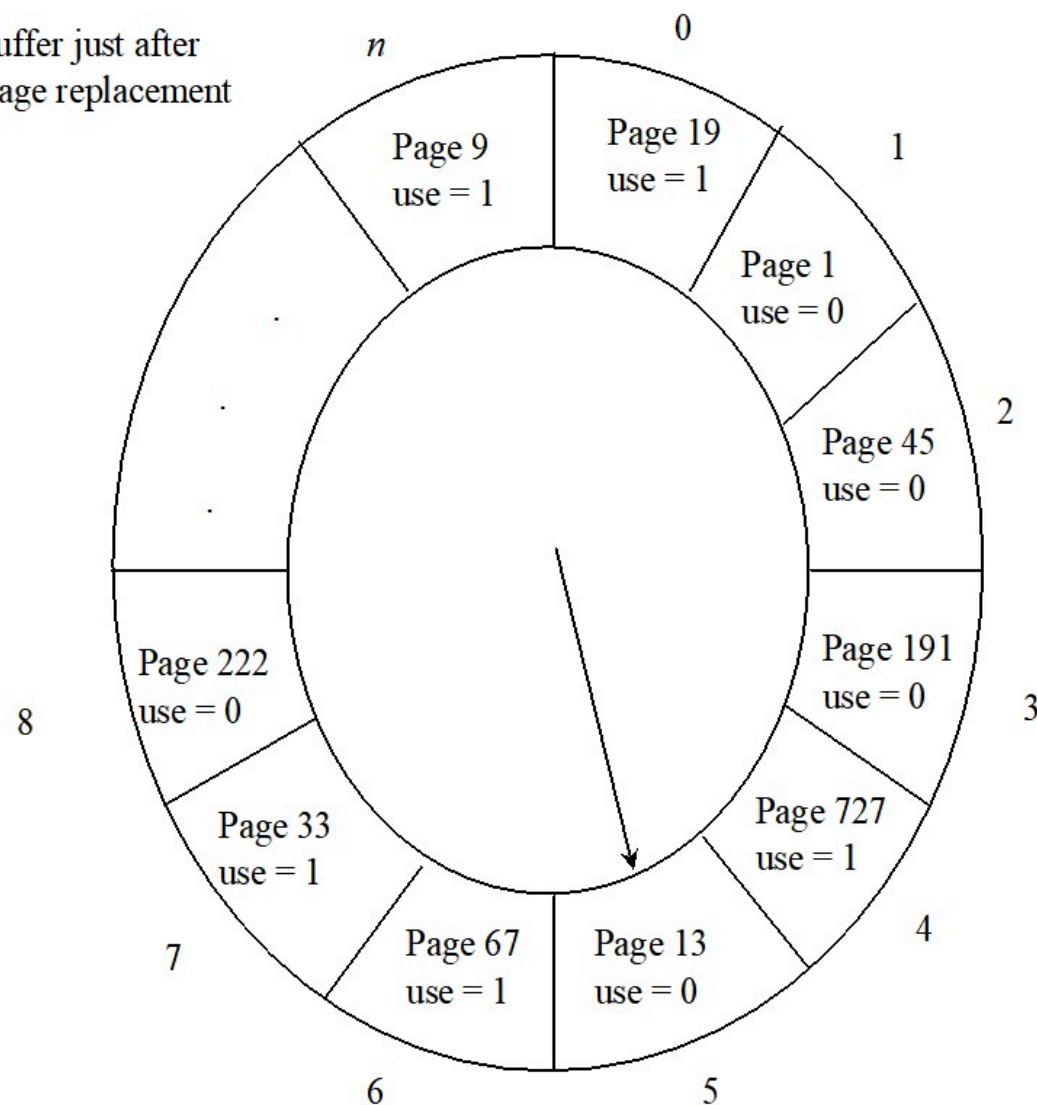


Part 2 请求分页存储管理

2.5 页面置换算法

State of buffer just after
the next page replacement

Clock置换算法示例



Part 2 请求分页存储管理

2.5 页面置换算法 改进Clock置换算法

除须考虑页面的使用情况外，还须增加置换代价

淘汰时，同时检查访问位A与修改位M

- **第1类** ($A=0, M=0$)：表示该页最近既未被访问、又未被修改，是最佳淘汰页。
- **第2类** ($A=0, M=1$)：表示该页最近未被访问，但已被修改，并不是很好的淘汰页。
- **第3类** ($A=1, M=0$)：表示该页最近已被访问，但未被修改，该页有可能再被访问。
- **第4类** ($A=1, M=1$)：表示该页最近已被访问且被修改，该页有可能再被访问。

置换时，循环依次查找第1类、第2类页面，找到为止

2.5 页面置换算法

最少使用置换算法LFU

为内存中的每个页面设置一个移位寄存器，用来记录该页面的被访问频率

LFU 选择在**最近时期使用最少的页面**作为淘汰页

2.5 页面置换算法

页面缓冲算法PBA

- ❖ 是对FIFO算法的发展，通过被置换页面的缓冲，有机会找回刚被置换的页面；
- ❖ 被置换页面的选择和处理：用FIFO算法选择被置换页，把被置换的页面放入两个链表之一。
 - 如果页面未被修改，就将其归入到空闲页面链表的末尾
 - 否则将其归入到已修改页面链表。

2.5 页面置换算法

页面缓冲算法PBA

- 需要调入新的物理页面时，将新页面内容读入到空闲页面链表的第一项所指的页面，然后将第一项删除。
- 空闲页面和已修改页面，仍停留在内存中一段时间，如果这些页面被再次访问，只需较小开销，而被访问的页面可以返还作为进程的内存页。
- 当已修改页面达到一定数目后，再将它们一起调出到外存，然后将它们归入空闲页面链表，这样能大大减少I/O操作的次数。

2.5 页面置换算法

● 置换算法举例

某程序在内存中分配三个页面，初始为空，页面走向为4, 3, 2, 1, 4, 3, 5, 4, 3, 2, 1, 5。

分别分析采用OPT, LRU, FIFO, CLOCK算法时的缺页情况和缺页率。

2.6 访问内存的有效时间

系统中有快表，快表时间 λ ，页表时间 t ，缺页处理时间 ε ，快表命中率 a ，缺页率 f

- (1) 快表命中
- (2) 快表未命中，不缺页
- (3) 快表未命中，缺页

分析请求分页存储管理的地址转换过程。

2.6 访问内存的有效时间

具有快表的请求分页管理方式的内存访问操作：

(1) 被访问页在内存中，对应页表项在快表中

$$EAT = \lambda + t$$

(2) 被访问页在内存中，对应页表项不在快表中

$$EAT = \lambda + t + \lambda + t = 2 \times (\lambda + t)$$

(3) 被访问页不在内存中

$$EAT = \lambda + t + \varepsilon + \lambda + t = 2 \times (\lambda + t) + \varepsilon$$

(4) 快表命中率为 a ，缺页率为 f ，缺页中断处理时间为 ε

$$EAT = \lambda + a \times t + (1 - a) \times ((1 - f) \times (\lambda + t) + f \times (t + \varepsilon + \lambda) + t)$$

CPU访存过程

Cache+虚拟存储

CPU 给出虚拟地址 VA

TLB缺失可由硬件也可由OS处理

Cache缺失由硬件处理

缺页由OS处理（缺页异常）

